
Software Requirements Specification

for

Rafeeq

Version 1.0

Prepared by *the WHOLE TEAM*

1/17/2026

Table of Contents

Table of Contents	2
Revision History.....	3
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience	1
1.4 Product Scope	2
2. Overall Description.....	3
2.1 Product Perspective	3
2.2 Product Functions	3
2.3 User Classes and Characteristics	4
2.4 Operating Environment	4
2.5 Design and Implementation Constraints.....	5
3. External Interface Requirements	6
3.1 User Interfaces	6
3.2 Hardware Interfaces.....	6
3.3 Software Interfaces	7
3.4 Communications Interfaces	8
4. User Stories & Acceptance Criteria	9
4.1 Epic: User Management.....	9
4.2 Epic: Place Discovery	11
4.3 Epic: Google Maps Integration & Navigation	12
4.4 Epic: Favorites (Wishlist)	14
4.5 Epic: Reviews & Ratings (Community)	15
4.6 Epic: Sponsors & Promotions.....	16
4.7 Epic: Itinerary & Trip Planner	17
4.8 Epic: Notifications & Alerts	18
4.9 Epic: Offline Mode & Caching.....	19
4.10 Epic: Admin & Content Management.....	20
4.11 Epic: Nearby & AI Recommendations.....	21
4.12 Epic: Multi-City & Regional Support.....	22
4.13 Epic: Events & Temporary Exhibitions	23
4.14 Epic: Transportation Enhancements	23
4.15 Epic: Emergency & Assistance Module	23
4.16 Epic: Weather & Travel Conditions	24
4.17 Epic: Content Reporting & Moderation.....	25
5. Non-functional Requirements	27
5.1 Performance Requirements.....	27
5.2 Safety Requirements.....	27
5.3 Security Requirements.....	28
5.4 Software Quality Attributes	28
5.5 Business Rules	29
6. System Models.....	30
6.1 Use Case Diagrams.....	30
6.2 Sequence Diagrams.....	31
5.3 Activity Diagrams	34
5.4 Class Diagrams	41

6.5 Entity–Relationship (ER) Diagram.....	43
---	----

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **Egypt Tourist Guide Mobile Application, Rafeeq, Version 1.0**. The purpose of this document is to provide a complete and detailed description of the system behavior, features, and constraints for stakeholders including developers, project supervisors, and evaluators.

This SRS covers the mobile application developed using **Flutter**, the backend services implemented using **ASP.NET Core Web API**, and the supporting database system. The scope includes all major subsystems such as user management, AI-driven place discovery, tourist place discovery, map and navigation services, trip planning, sponsor management, and administration modules.

This document serves as a reference for system design, implementation, testing, and validation throughout the development lifecycle of the graduation project.

1.2 Document Conventions

The following conventions are used throughout this document:

- **Functional Requirements:**
 - **US-xx:** User Story ID (for Agile tracking).
- **Non-Functional Requirements:**
 - **PRx.y.z:** Performance Requirements.
 - **SaR:** Safety Requirements.
 - **SecR:** Security Requirements.
 - **QA:** Quality Attributes (Usability, Reliability, Maintainability, Portability).
 - **ScalR:** Scalability Requirements.
 - **BR:** Business Rules.
- The keyword “**shall**” is used to indicate a mandatory requirement.
- Important terms and section titles are written in **bold**.

All requirements are written in clear, unambiguous language.

1.3 Intended Audience

This document is intended for the following audiences:

- **Project Supervisors and Examiners:** To understand the system scope, objectives, and evaluation criteria.
- **Developers:** To implement the mobile application, backend services, and database according to specified requirements.

- **Documentation Writers:** To prepare user manuals and technical guides.

The document is organized as follows:

1. **Introduction:** Provides an overview of the system, scope, and objectives.
2. **Overall Description:** Describes system context, user classes, and operating environment.
3. **External interface requirements:** Includes User, Hardware, Software, Communication Interfaces Requirements.
4. **User Stories / Functional Requirements:** Details all system features and behaviors.
5. **Non-Functional Requirements:** Defines quality attributes such as performance, security, and usability.
6. **System Models:** Includes use case diagrams, sequence diagrams, and ER diagrams.

1.4 Product Scope

The **Rafeeq Application** is a location-based mobile system designed to assist tourists in discovering, exploring, and navigating historical and cultural sites across Egypt. The system provides detailed information about attractions, interactive maps with routing and transportation options, trip planning tools, AI-based recommendations, user reviews, sponsor recommendations, and emergency assistance.

The system encompasses:

- **Discovery:** Detailed information on historical sites, temporary exhibitions, and events.
- **Navigation:** Interactive Google Maps integration with multi-modal routing (driving, walking, transit).
- **Planning:** Tools for creating single-day or multi-city trips with route optimization.
- **Safety & Context:** Real-time weather alerts, emergency SOS services, and embassy contact integration.
- **Engagement:** A community system for reviews, ratings, and content reporting.
- **Monetization:** A platform for sponsors to offer time-limited discounts to tourists.

The primary objectives of the system are:

- To enhance the tourist experience by providing accurate, up-to-date, and accessible information.
- To enable easy navigation and route planning using Google Maps services.
- To support multi-language content for international users.
- To offer personalized recommendations and itinerary planning.
- To provide a scalable platform that can integrate sponsors and tourism services.

This application supports Egypt's digital tourism strategy by promoting cultural heritage, improving tourist accessibility, and enabling data-driven tourism services.

2. Overall Description

2.1 Product Perspective

The **Egypt Tourist Guide Mobile Application** is a new, self-contained software product developed as a graduation project. It is not a direct replacement for an existing system, but rather an integrated digital solution that combines location services, tourism information systems, and mobile navigation technologies into a single platform.

The system consists of three main components:

1. **Mobile Application (Flutter):** Provides the user interface for tourists, including place browsing, maps, trip planning, and notifications.
2. **Backend Server (ASP.NET Core Web API):** Handles business logic, user authentication, data management, recommendation services, sponsor management, and integration with external APIs.
3. **Admin Dashboard (Web-based):** A management portal for Administrators and Moderators to manage places, events, sponsors, users, and review reported content.

External Service Integrations:

- **Google Maps Platform:** For maps, places, routing, and traffic data.
- **Weather APIs:** For real-time temperature and severe weather alerts.
- **Firebase Cloud Messaging (FCM):** For push notifications regarding trips, offers, and emergencies.

The mobile application communicates with the backend through RESTful APIs, and the backend integrates with external services to provide real-time maps, routes, and location-based information.

2.2 Product Functions

At a high level, the system provides the following major functions:

- User registration, authentication, and profile management
- Browsing and searching historical and cultural places in Egypt
- Viewing detailed place information, images, and multimedia
- Displaying places and routes on interactive Google Maps
- Generating directions and transportation options
- Trip planning and itinerary creation
- Nearby place recommendations based on user location
- User reviews, ratings, and favorites
- Sponsor and partner promotion
- Push notifications and alerts
- Emergency assistance and important contacts

- Administrative content management and moderation
- Analytics and reporting for system administrator

2.3 User Classes and Characteristics

The system supports the following user classes:

1. Tourist (Guest)

- General public users, both local and international.
- Limited technical knowledge.
- Use the mobile application to explore places, navigate, plan trips, and read reviews.

2. Tourist (Registered)

- Same as tourist but with an account.
- Can save favorites, write reviews, create trips, receive notifications

3. Administrator

- Technically trained personnel.
- Full system privileges.
- Manage places, sponsors, users, reports, and system configuration.

4. Moderator

- Limited administrative role.
- Responsible for content approval, review moderation, and user reports.

2.4 Operating Environment

The system shall operate in the following environment:

- **Mobile Platforms:**
 - Android 9.0+
 - iOS 13+
 - Requires GPS and Internet (Wi-Fi/4G/5G).
- **Backend Server:**
 - ASP.NET Core (.NET 10) hosted on cloud infrastructure (e.g., Azure/AWS).
- **Database:**
 - SQL Server or PostgreSQL
- **Web Admin Portal:**
 - Accessible via modern web browsers (Chrome, Edge, Safari).

2.5 Design and Implementation Constraints

The system is subject to the following constraints:

- The mobile application must be developed using **Flutter**.
- The backend must be developed using **ASP.NET Core**.
- The system must integrate with **Google Maps Platform** for location and routing services.
- The database must be a relational database system.
- All communication must use HTTPS.
- The system must support English and Arabic languages.
- Development must follow Clean Architecture and RESTful API design principles.
- The project timeline and resources are constrained by academic semester duration.
- The application must comply with mobile platform privacy and location permission policies.

3. External Interface Requirements

3.1 User Interfaces

The system shall provide a mobile-based graphical user interface designed for tourists using smartphones. The user interface will be developed using **Flutter**, ensuring a consistent look and feel across Android and iOS platforms.

The application shall include the following user interface components:

- **Home Screen:** Displays featured historical places, sponsored places, and navigation options.
- **Place Listing Screen:** Displays a list of historical places with images, names, and cities.
- **Place Details Screen:** Displays detailed information about a selected place, including description, images, opening hours, ticket prices, and location on Google Maps.
- **Map Screen:** Displays the selected place and route directions using Google Maps.
- **Favorites Screen:** Displays places bookmarked by the user.
- **Trip Planner Screen:** Allows users to create and manage itineraries.
- **User Profile Screen:** Allows users to manage account details and language preferences.
- **Admin Dashboard (Web-based):** Allows administrators to manage places, sponsors, users, and reviews.

The user interface shall follow these standards and constraints:

- The interface shall support **English and Arabic languages**.
- The interface shall use standard mobile navigation components such as bottom navigation bars and icons.
- Consistent buttons (Back, Save, Cancel) shall be used throughout the application.
- Error messages shall be displayed in a clear and user-friendly manner.
- Confirmation messages shall be shown for critical actions (e.g., deleting favorites, submitting reviews).

3.2 Hardware Interfaces

The system shall operate on standard smartphone hardware and cloud-based server infrastructure.

Mobile Devices:

- Smartphones running **Android** or **iOS**
- **GPS** sensor for location-based services to determine the user's current coordinates (Latitude/Longitude).
- **Camera** and access **photo gallery** to allow users to upload profile pictures or attach photos to reviews.
- **Storage** to cache images and user preferences (approx. 100MB max cache size).
- Internet connectivity via Wi-Fi or mobile data

Server Hardware:

- Cloud-based server hosting the ASP.NET Core API
- Database server for storing application data

The application shall interact with hardware components as follows:

- GPS hardware shall be used to obtain the user's current location.
- Network hardware shall be used to communicate with backend services and third-party APIs.

No direct low-level hardware control is required by the system.

3.3 Software Interfaces

The system shall interface with the following software components:

Backend:

- **ASP.NET Core Web API (.NET 10)**
- **SQL Server** database for persistent data storage

Frontend:

- **Flutter Framework** for mobile application development

Third-Party Services:

- **Google Maps API** for map display, location services, and route generation
- **Google Directions API** for route calculation and distance estimation
- **Firestore Cloud Messaging (FCM):** Used for delivering push notifications to the mobile client.
- **Authentication Services** (Email / Google Sign-In)

Operating Systems:

- Android OS
- iOS
- Windows or Linux (server environment)

Data Exchange:

- Communication between Flutter app and backend API shall use **RESTful services**.

- Data shall be exchanged in **JSON format**.
- All protected endpoints require a Bearer Token (JWT) in the HTTP Authorization header.
- The backend shall provide endpoints for places, users, reviews, sponsors, and trips.
- Shared data includes user profiles, place information, reviews, and sponsor details.

All API communication shall follow REST standards and be documented using **OpenAPI (Swagger)**.

3.4 Communications Interfaces

The system shall support communication over standard internet protocols.

- The application shall use **HTTP/HTTPS** for all client-server communication.
- All sensitive data transmissions shall use **HTTPS encryption**.
- The system shall communicate with external services such as Google Maps via secure API calls.
- JSON shall be used as the standard message format for all communications.
- The system shall support real-time location data retrieval for route generation.

Security requirements:

- API access shall be protected using authentication tokens.
- User data shall not be transmitted in plain text.
- The system shall prevent unauthorized access to admin functionalities.

The system does not require support for email, FTP, or peer-to-peer communication in its current scope.

4. User Stories & Acceptance Criteria

This section organizes the main system features of the *Rafeeq* into **Epics**, where each Epic represents a high-level system capability. Each user story represents a functional requirement from the perspective of an end user and defines how the system should behave to deliver value. Each Epic is decomposed into user stories, and each user story is supported by acceptance criteria to ensure clarity, completeness, and testability.

4.1 Epic: User Management

4.1.1 Authentication & Security

US-1 User Registration

As a new tourist, **I want** to create an account using my email or Google account, **So that** I can access personalized features like saving trips.

Acceptance Criteria:

- User can sign up by entering a valid email and password.
- User can sign up using the "Continue with Google" button (OAuth).
- System validates that the email format is correct and not already in use.
- System creates a record in the database upon success.

US-2 Secure Login & Logout

As a registered user, **I want** to log in and log out securely, **So that** I can access my account on different devices without unauthorized access.

Acceptance Criteria:

- User can log in with registered credentials.
- Failed login attempts display a generic error message (e.g., "Invalid credentials") for security.
- Clicking "Logout" invalidates the current session immediately.
- User is redirected to the home/login screen after logout.

US-3 Password Reset

As a user who forgot their password, **I want** to reset it via email, **So that** I can recover access to my account.

Acceptance Criteria:

- User can request a reset link by providing their registered email.
- System sends a time-sensitive link to the email.
- Clicking the link allows the user to set a new password.
- Old password is no longer valid after the reset.

4.1.2 Profile & Preferences

US-4 Edit Profile Details

As a registered user, **I want** to edit my personal details (name, email), **So that** my profile information is always up to date.

Acceptance Criteria:

- User can update their Display Name and Email.
- Changes are saved to the database immediately upon clicking "Save."
- Email changes require re-verification (optional but recommended).

US-5 Localization & Units (Language & Distance)

As an international tourist, **I want** to select my preferred language and distance units, **So that** the app content is understandable and relevant to me.

Acceptance Criteria:

- User can toggle language between English and Arabic.
- User can toggle distance units between Kilometers (km) and Miles (mi).
- The UI updates immediately to reflect the chosen language and units.
- Preferences persist across login sessions.

US-6 Notification Settings

As a user, **I want** to turn push notifications on or off, **So that** I am not disturbed when I don't want to be.

Acceptance Criteria:

- User can view a "Notifications" toggle in settings.
- Setting to "Off" blocks all marketing/non-essential alerts.
- Setting "On" enables alerts.

US-7 Delete Account

As a privacy-conscious user, **I want** to permanently delete my account, **So that** my personal data is removed from the system.

Acceptance Criteria:

- User must confirm the deletion action (e.g., a "Are you sure?" popup).
- System performs a "soft delete" or permanent purge (depending on data policy) of user PII (Personally Identifiable Information).
- User is logged out and redirected to the welcome screen.

4.1.3 User History & Roles

US-8 View Activity History

As a frequent traveler, **I want** to view my past activity, **So that** I can see places I've visited and reviews I've written.

Acceptance Criteria:

- User can access a "History" or "My Activity" tab.
- The view displays a list of:
 - Places marked as "Visited."
 - Reviews submitted.
 - Trips created.
- Items in the list are clickable and navigate to the respective details.

US-9 Role-Based Access Control (RBAC)

As a System Admin, **I want** to assign roles (User, Moderator, Admin) to accounts, **So that** users only have access to features permitted for their level.

Acceptance Criteria:

- **Standard Users** can only edit their own data.
- **Moderators** can hide/delete inappropriate reviews but cannot delete users.
- **Admins** have full access to all system configurations and user management.
- The system checks the user's role token before loading protected pages.

4.2 Epic: Place Discovery

4.2.1 Browsing & Search

US-10 Browse Historical Places

As a tourist, **I want** to view a list of historical places in Egypt, **So that** I can discover interesting locations to visit.

Acceptance Criteria:

- The home screen displays a scrollable list (or grid) of places.
- Each list item shows a thumbnail image and the place name.
- Pagination or "infinite scroll" is implemented if there are more than 20 places.

US-11 Search and Filter Places

As a user who knows what they are looking for and have specific preferences, **I want** to search (by name or keyword) and filter (by City, Category, etc.) places, **So that** I can quickly find specific attractions.

Acceptance Criteria:

- Search bar is visible at the top of the places screen.
- Results update as I type (live search) OR upon clicking "Search."
- If no results are found, display a "No places found" message.
- User can select filters from a menu (e.g., City: "Luxor", Type: "Temple").
- User can sort results by "Popularity" (High to Low).
- The list updates immediately to show only matching items.
- "Clear All" button resets the list to default.

4.2.2 Place Information**US-12 View Place Details**

As a user, **I want** to see detailed information about a specific place (images, price, hours), **So that** I can decide if it fits my schedule and budget.

Acceptance Criteria:

- Tapping a place in the list opens a Details Page.
- The page displays: Name, Description, Location, Ticket Price, and Opening Hours.
- A gallery or carousel allows swiping through multiple images.
- Contact details (phone/website) are clickable.

4.3 Epic: Google Maps Integration & Navigation**4.3.1 Map Visualization & Interaction****US-13 View Place on Map**

As a traveler, **I want** to view the exact location of a place on an interactive map, **So that** I can understand its surroundings.

Acceptance Criteria:

- User can click "View on Map" from the Details page.
- The map opens with a pin dropped on the specific latitude/longitude.
- Tapping the pin displays the place name.

US-14 Map Controls (Zoom & Layers)

As a user navigating the map, **I want** to zoom in/out and switch between Map and Satellite views, **So that** I can see the terrain or street details clearly.

Acceptance Criteria:

- Standard pinch-to-zoom gestures work smoothly.
- A toggle button switches the layer between "Default" (Vector) and "Satellite".
- Controls are positioned so they don't obscure the map center.

US-15 Real-Time Traffic

As a driver, **I want** to see real-time traffic conditions overlayed on the map, **So that** I can avoid congested areas.

Acceptance Criteria:

- User can toggle a "Traffic" layer on/off.
- Roads are color-coded (Green = Clear, Red = Heavy Traffic) based on Google Maps data.

4.3.2 Routing & Navigation

US-16 Route Planning & Transport Modes

As a traveler, **I want** to generate a route from my current location and choose my transport mode (Driving, Walking, Transit), **So that** I can estimate my travel time.

Acceptance Criteria:

- System detects the user's current GPS location as the "Start" point.
- User can switch tabs for Driving, Walking, or Public Transit.
- System displays the route line on the map.
- System displays estimated **Time** and **Distance** for the selected mode.

US-17 Alternative Routes

As a user, I want to see multiple route options (if available), **So that** I can choose the fastest or most scenic path.

Acceptance Criteria:

- If Google APIs return multiple routes, they are displayed as grayed-out alternative lines.
- Tapping an alternative line makes it the active route and updates the time/distance estimates.

US-18 Dynamic Navigation (Rerouting)

As a driver, I want the route to update automatically if I deviate from the path, **So that** I can get back on track without manual input.

Acceptance Criteria:

- App tracks user movement in real-time.
- If the user leaves the calculated path, the system triggers a re-calculation (Rerouting).
- *Note: This usually requires specific Navigation SDK implementation.*

US-19 Offline Route Access

As a user with spotty internet, I want to save a generated route, **So that** I can view the path even when I go offline.

Acceptance Criteria:

- User can click a "Save Route" button after a route is generated.
- The polyline (route path) is cached locally.
- When offline, the map still renders the route line (even if background map tiles are missing).

4.4 Epic: Favorites (Wishlist)

4.4.1 Manage Favorites

US-20 Toggle Favorite Status

As a user browsing places, I want to mark a place as a "Favorite" (or remove it), **So that** I can easily find it later without searching again.

Acceptance Criteria:

- A "Heart" icon is visible on both the list view and details page.
- Tapping the empty heart fills it (Add to Favorites) and triggers a brief success message (toast).
- Tapping the filled heart empties it (Remove from Favorites).
- The status updates immediately in the database.

US-21 View Favorites List

As a user planning a trip, **I want** to view a dedicated list of all my favorite places, **So that** I can create an itinerary from my shortlisted locations.

Acceptance Criteria:

- A "Favorites" tab or menu item is accessible from the main navigation.
- The screen displays only the places marked as favorite by the current user.
- Clicking a place navigates to its Details Page.
- If the list is empty, display a prompt: "You haven't saved any places yet."

4.5 Epic: Reviews & Ratings (Community)

4.5.1 Manage Reviews & Ratings

US-22 Submit Review & Rating

As a registered user who has visited a place, **I want** to submit a star rating and a text review, **So that** I can share my experience with others.

Acceptance Criteria:

- User can select a rating from 1 to 5 stars (mandatory).
- User can enter text comments (optional or mandatory, depending on your rule).
- User can click "Submit" to publish the review.
- System validates that the user is logged in before allowing submission.

US-23 View Place Reviews

As a potential visitor, **I want** to read reviews and see the average rating of a place, **So that** I can judge if it is worth visiting.

Acceptance Criteria:

- The Details Page displays the average rating (e.g., 4.5/5) and total review count.
- A list of individual reviews is displayed below the details.
- Each review shows:
 - Username (or "Anonymous" if the user chose to hide their name).

- Star Rating.
- Text Content.
- Date of submission (e.g., "2 days ago").

4.6 Epic: Sponsors & Promotions

4.6.1 Sponsored Place Visibility & Information

US-24 View Sponsored Places (Priority Listing)

As a user searching for recommendations, **I will** see sponsored places appearing near the top of my list, **So that** I am exposed to premium partners (hotels/restaurants).

Acceptance Criteria:

- Sponsored places appear within the top 3-5 results in the list view.
- These items are clearly marked with a "Sponsored" or "Ad" tag to distinguish them from organic results.
- The ranking algorithm prioritizes these items over standard places in the default sort order.

US-25 View Sponsor Details

As a user interested in a sponsored service, **I want** to view their specific details (Website, Logo, Contact), **So that** I can book a service or learn more.

Acceptance Criteria:

- Clicking a sponsored place opens a specialized Details Page.
- The page prominently displays the Sponsor's Logo and Website link.
- A "Contact" button allows direct dialing or emailing.

4.6.2 Promotional Offers & Discounts

US-26 Time-Limited Offers

As a budget-conscious traveler, **I want** to see special time-limited discounts from sponsors, **So that** I can save money on my trip.

Acceptance Criteria:

- If a sponsor has an active campaign, a "Special Offer" badge is visible on their listing (e.g., "20% Off").
- The Details Page displays the offer terms and the expiration date/time.

- Expired offers are automatically hidden from the view.

4.7 Epic: Itinerary & Trip Planner

4.7.1 Trip Management

US-27 Create and Manage Trips

As a user planning a vacation, **I want** to create a new trip by adding multiple places, **So that** I can organize my visit in one list.

Acceptance Criteria:

- User can create a named trip (e.g., "Cairo Weekend").
- User can add places to the trip from the "Details" page or "Favorites" list.
- User can edit the trip (add/remove places) or delete the entire trip.
- Trips are saved to the user's profile.

US-28 Trip Estimates

As a user, **I want** to see the total estimated duration and travel time for my itinerary, **So that** I know if my plan is realistic for one day.

Acceptance Criteria:

- System calculates total estimated travel time between points.
- System adds estimated "Time Spent" at each location (e.g., default 1 hour per museum) to the total duration.
- The UI displays "Total Trip Time: X hours."

4.7.2 Route Planning & Optimization

US-29 Route Optimization

As a traveler visiting multiple spots, **I want** the system to suggest the best order to visit them, **So that** I save travel time and avoid backtracking.

Acceptance Criteria:

- System analyzes the selected locations.
- System reorders the list to minimize total travel distance (a simple "Traveling Salesman" logic).
- User can manually override the suggested order by dragging and dropping items.

4.8 Epic: Notifications & Alerts

4.8.1 Personal Updates

US-30 Trip Reminders

As a traveler, **I want** to receive reminders for my upcoming trips, **So that** I don't forget my schedule.

Acceptance Criteria:

- Push notification is sent 24 hours (or configurable time) before a scheduled trip.
- Tapping the notification opens the Trip Planner view.

US-31 Place Updates (Favorites)

As a user, **I want** to be notified if a place I favorited has new hours or closures, **So that** I don't show up to a closed door.

Acceptance Criteria:

- System monitors changes to "Opening Hours" or "Status" of favorite places.
- Notification is sent: "Alert: The Egyptian Museum is closed for renovation."

4.8.2 Marketing & Context

US-32 Promotional & Contextual Alerts

As a user, **I want** to receive offers or recommendations relevant to where I am, **So that** I can find good deals nearby.

Acceptance Criteria:

- **Geo-fencing:** When a user enters a specific city/zone, send a recommendation (e.g., "You are near Luxor Temple!").
- **Promos:** Send alerts for active sponsor discounts.
- **Opt-Out:** Check user preferences (from US-6) before sending. If "Sponsored Notifications" is OFF, do not send.

4.8.3 System & Safety

US-33 Safety & System Alerts

As a user, I want to receive critical alerts about weather hazards or app maintenance, **So that** I can stay safe and informed.

Acceptance Criteria:

- **Emergency:** Send high-priority alerts for local hazards (e.g., "Sandstorm warning in Giza").
- **System:** Send alerts for downtime (e.g., "App maintenance scheduled for 2 AM").
- These critical alerts may bypass standard "Do Not Disturb" settings if allowed by OS permissions.

4.9 Epic: Offline Mode & Caching

4.9.1 Offline Data & Caching

US-34 Automatic Caching (Recent History)

As a user with intermittent internet, **I want** to view details of places I recently looked at even when offline, **So that** I don't lose access to information I just found.

Acceptance Criteria:

- System automatically caches the JSON data and images for the last X viewed places.
- When offline, these places load from local storage instead of the API.
- If data is not in cache, show a "No Connection" placeholder.

US-35 Offline Favorites

As a proactive traveler, I want my "Favorite" places to always be available offline, **So that** I can access my most important info anytime.

Acceptance Criteria:

- Marking a place as "Favorite" triggers a download of its full details (Description, Images, basic Metadata).
- Favorites list is fully accessible without internet.

4.9.2 Connectivity Awareness & Feedback

US-36 Connectivity Feedback

As a user, I want the app to warn me if I try to use an online-only feature while offline, **So that** I understand why it's not working.

Acceptance Criteria:

- App detects network status changes.
- If offline user tries to route (Google Maps) or search fresh data, show a Toast/Snackbar: "No Internet connection. Some features are unavailable."
- Map view shows a "Offline Mode" banner if live traffic/tiles cannot load.

4.10 Epic: Admin & Content Management**4.10.1 Content & Sponsor Management****US-37 Manage Places (CRUD)**

As an Admin, I want to add, edit, or delete tourist places, **So that** the app content remains accurate and up-to-date.

Acceptance Criteria:

- Admin can fill a form to create a new place (Name, Description, Coordinates, Images).
- Admin can search for an existing place and update its details (e.g., change ticket price).
- Admin can "Soft Delete" a place (mark as inactive) so it doesn't break old user trips.

US-38 Moderate Reviews

As a Moderator/Admin, I want to approve or reject user reviews, **So that** I can prevent spam or inappropriate content from appearing.

Acceptance Criteria:

- Admin sees a "Pending Reviews" queue.
- Admin can "Approve" (publish) or "Reject" (delete) a review.
- Rejected reviews are removed from the public view but kept in an archive for audit.

US-39 Manage Sponsors & Offers

As an Admin, I want to manage sponsor profiles and their active offers, **So that** monetization partnerships are handled correctly.

Acceptance Criteria:

- Admin can link a Sponsor profile to a specific Place ID.
- Admin can set start/end dates for promotional offers.
- Admin can upload sponsor logos and contact info.

4.10.2 User Management & Audit

US-40 User Management & Moderation

As an Admin, I want to manage user accounts (block, suspend), **So that** I can maintain a safe community.

Acceptance Criteria:

- Admin can search for users by email or name.
- Admin can change a user's status to "Active," "Suspended," or "Banned."
- Admin can flag accounts as "Suspicious" for further review.

US-41 Audit Logs

As a Super Admin, I want to view a log of all administrative actions, **So that** I have accountability for changes made to the system.

Acceptance Criteria:

- A read-only "Audit Log" table displays: Timestamp, Admin Username, Action (e.g., "Deleted Place X"), and IP Address.
- Logs cannot be deleted by standard admins.

4.11 Epic: Nearby & AI Recommendations

4.11.1 Location-Based Discovery

US-42 View Nearby Places

As a tourist on the go, I want to see places near my current location, **So that** I can find something to do right now without traveling far.

Acceptance Criteria:

- App requests GPS permission.
- List automatically filters to show items within X km (configurable radius, e.g., 5km).
- Results are sorted by "Distance: Closest First."

US-43 Popular Nearby (Highlight)

As a user, I want to see which nearby places are most popular, **So that** I don't miss the main attractions.

Acceptance Criteria:

- A "Trending Nearby" section appears at the top of the nearby list.
- It displays the top 3 highest-rated places within the user's radius.

4.11.2 Intelligent Suggestions**US-44 AI-Based Suggestions**

As a user, I want personalized recommendations based on my history, **So that** I discover places I will actually like.

Acceptance Criteria:

- **Algorithm:** "If user liked 'History Museums', suggest 'Temples'."
- **Implementation:** A "Recommended for You" section on the home screen.
- Recommendations update as the user favorites more places.

4.11.3 Alerts**US-45 Operational Alerts (Nearby Status)**

As a user, I want to be warned if a nearby place is closed, **So that** I don't waste time walking there.

Acceptance Criteria:

- If a nearby place's "Opening Hours" indicate it is currently Closed, display a "Closed Now" tag in red on the list card.

4.12 Epic: Multi-City & Regional Support**4.12.1 City/Region Selection & Multi-City Trips****US-46 Select Region/City**

As a user planning a trip, I want to switch the app view to a specific city (e.g., Luxor), **So that** I only see places relevant to that area.

Acceptance Criteria:

- A "Location/City" dropdown is available in the top bar.
- Selecting "Luxor" filters the Home Feed and Map View to show only Luxor data.
- "All Egypt" is the default view.

US-47 Cross-City Trip Planning

As a traveler on a long vacation, **I want** to build an itinerary that includes stops in multiple cities (e.g., Cairo -> Aswan), **So that** I can manage my entire vacation in one trip file.

Acceptance Criteria:

- The Trip Planner (US-27) allows adding places from different cities into the same trip.
- The Route Optimizer (US-28) handles long-distance routing between cities intelligently (e.g., grouping all Cairo stops before moving to Aswan stops).

4.14 Epic: Transportation Enhancements

4.14.1 Route Comparison

US-48 Route Comparison

As a traveler, **I want** to compare different route options side-by-side (Time vs. Distance), **So that** I can choose the most efficient path.

Acceptance Criteria:

- System displays multiple route cards (e.g., "Fastest", "Shortest", "Less Traffic").
- User can tap each option to visualize the path on the map.

4.14.2 Public Transportation Discovery

US-49 Nearby Public Transport

As a backpacker relying on public transit, **I want** to see nearby metro and bus stations on the map, **So that** I can find my way to the site without a car.

Acceptance Criteria:

- User can toggle a "Transit" layer on the map.
- System highlights nearby Metro stations and Bus stops using distinct icons.
- Tapping a station shows its name and served lines.

4.15 Epic: Emergency & Assistance Module

4.15.1 Emergency Service & Assistance

US-50 Emergency Assistance Hub (SOS)

As a tourist in distress, **I want** a quick-access "Emergency" button, **So that** I can contact police or ambulance immediately.

Acceptance Criteria:

- A persistent or easily accessible "SOS" or "Help" button is available in the menu.
- Tapping it opens a screen with one-tap dial buttons for:
 - Tourist Police (126).
 - Ambulance (123).
 - Tourism Hotline (19654).
- **Dynamic Feature:** If the user has set their Nationality in their profile, display the contact number for their local Embassy.

US-51 Locate Nearby Services

As a user, **I want** to view nearby hospitals and police stations on the map, **So that** I know where to go in case of a non-critical emergency.

Acceptance Criteria:

- User can select "Hospitals" or "Police" from the "Find Nearby" filter.
- Map pins for these services are colored differently (e.g., Red for Medical, Blue for Police).
- Tapping a pin allows instant navigation (Route) to that location.

4.16 Epic: Weather & Travel Conditions

4.16.1 Weather Information & Alerts

US-52 View Weather Conditions

As a user planning a visit, **I want** to see the current temperature and conditions for a specific place, **So that** I dress appropriately.

Acceptance Criteria:

- Place Details page displays a weather widget (e.g., "35°C, Sunny").
- Data is fetched from a Weather API based on the place's coordinates.

US-53 Extreme Weather Alerts

As a user, **I want** to receive alerts about sandstorms or heat waves, **So that** I can avoid outdoor sites during dangerous conditions.

Acceptance Criteria:

- System checks daily weather forecasts for "Severe" warnings.
- If a warning exists for the user's location, a red banner or push notification is triggered: "Heat Wave Warning: Avoid open areas."

4.16.2 Smart Visit Recommendations

US-54 Smart Visiting Hours (Crowd/Weather)

As a user, I want to know the best time to visit, **So that** I avoid the hottest sun or the biggest crowds.

Acceptance Criteria:

- System suggests "Recommended Time" (e.g., "Visit after 4 PM to avoid heat").
- (Optional AI/Data feature): Show a "Crowd Meter" (e.g., "Usually busy at 10 AM").

4.17 Epic: Content Reporting & Moderation

4.17.1 User Reporting & Moderator Review

US-55 Report Inappropriate Content

As a community member, I want to report abusive reviews or incorrect images, **So that** the platform remains safe and accurate.

Acceptance Criteria:

- A "Flag" or "Report" icon is visible on every Review and Place.
- User must select a reason (e.g., "Spam," "Offensive," "Incorrect Info").
- Submitting hides the content locally for that user and sends a flag to the Admin Dashboard.

US-56 Review Reported Content (Admin)

As a Moderator, I want to see a queue of reported items, **So that** I can take action (Delete or Keep).

Acceptance Criteria:

- Dashboard shows a "Reported Items" list.
- Admin can "Delete Content" (removes it for everyone) or "Dismiss Report" (keeps content).
- If content is deleted, the offending user receives a warning notification.

4.17.2 Moderation Accountability & Audit

US-57 Moderation Audit Log

As a System Admin, **I want** all moderation decisions to be logged, **So that** we can review if a moderator is abusing their power.

Acceptance Criteria:

- Every "Delete" or "Ban" action is recorded in the `ModerationLog` table.
- Log includes: Moderator ID, Action, Reason, and Timestamp.

5. Non-functional Requirements

This section defines the quality attributes, performance constraints, and safety regulations that the system must satisfy to ensure reliable and secure user experience.

5.1 Performance Requirements

The system shall meet the following performance requirements to ensure acceptable response time, throughput, and efficient resource usage for mobile users and backend services.

PR5.1.1 The system shall load the home screen within 3 seconds on a standard mobile network.

PR5.1.2 The system shall return search results within 2 seconds for up to 10,000 places.

PR5.1.3 The system shall display map tiles and routes with a response time not exceeding 2 seconds after user interaction.

PR5.1.4 The backend API shall support at least 1,000 concurrent users without degradation of service.

PR5.1.5 The system shall cache frequently accessed data (popular places, cities, categories) to reduce response time.

5.2 Safety Requirements

The system shall be designed to prevent any situation that may cause physical, financial, or informational harm to users during its operation.

SaR5.2.1 The system shall display clear warnings that navigation and map information are for guidance only and that users must follow official traffic rules and on-site safety instructions.

SaR5.2.2 The system shall not provide unsafe or restricted routes (e.g., closed roads, military zones) when such information is available from the map service provider.

SaR5.2.3 The system shall require user confirmation before enabling location tracking.

SaR5.2.4 The system shall protect personal and location data to prevent misuse or unauthorized access.

SaR5.2.5 The system shall provide emergency contact information (police, ambulance, tourism hotline) that can be accessed with one action from any screen.

SaR5.2.6 The system shall display safety alerts for dangerous weather conditions, natural hazards, or temporary site closures when such data is available.

SaR5.2.7 The system shall prevent sponsors or third parties from displaying misleading or unsafe information.

SaR5.2.8 The system shall comply with mobile application safety and data protection regulations, including user consent and privacy policies.

SaR5.2.9 The system shall ensure that no critical function (such as emergency contact access) is blocked by advertisements or sponsored content.

5.3 Security Requirements

SecR5.3.1 All communication between the mobile application and backend shall use HTTPS.

SecR5.3.2 User authentication shall be implemented using secure token-based authentication (JWT).

SecR5.3.3 User passwords shall be stored in encrypted and hashed form.

SecR5.3.4 The system shall enforce role-based access control (User, Admin).

SecR5.3.5 The system shall protect against common web attacks (SQL Injection, XSS, CSRF).

SecR5.3.6 Sensitive data (API keys, tokens) shall not be stored in plain text in the client application.

5.4 Software Quality Attributes

The software shall satisfy the following quality attributes:

Usability

QA5.4.1 The system shall allow a first-time user to search for a place and view its directions within three interactions.

QA5.4.2 The system shall support Arabic and English with clear, readable fonts and consistent layout.

Reliability

QA5.4.3 The system shall operate continuously with an uptime of at least 99% excluding maintenance.

QA5.4.4 The system shall recover automatically from application crashes without loss of user data.

QA5.4.5 The system shall ensure data consistency between the mobile application and backend database.

QA5.4.6 The system shall automatically retry failed API requests due to temporary network issues.

QA5.4.7 The system shall prevent duplicate entries of places and sponsors.

QA5.4.8 The system shall validate all input data before saving to the database.

Maintainability

QA5.4.9 The backend system shall follow **Clean Architecture principles**, separating the solution into distinct layers:

- Presentation Layer (API Controllers)
- Application Layer (Use Cases)
- Domain Layer (Entities, Value Objects, Interfaces)
- Infrastructure Layer (Database, External APIs, Google Maps, Notification Services)

QA5.4.10 The system shall ensure that changes in external services (e.g., Google Maps API, database provider, notification service) do not affect the core business logic.

QA5.4.11 The system shall include clear project structure and coding standards to allow new developers to understand and extend the system within a short onboarding time.

Portability

QA5.4.12 The system shall run on Android and iOS without code changes to core business logic.

Scalability

QA5.4.13 The system architecture shall support horizontal scaling of the API services.

QA5.4.14 The database design shall support growth to at least 1 million users and 100,000 places.

QA5.4.15 The system shall allow new cities and languages to be added without major redesign.

Interoperability

QA5.4.16 The system shall integrate with external services such as Google Maps and weather APIs using standard REST interfaces.

5.5 Business Rules

BR5.5.1 Only administrators shall be allowed to add, edit, or delete tourist places and cities.

BR5.5.2 Only moderators shall be allowed to approve, reject, or remove user reviews and reported content.

BR5.5.3 Only registered users shall be allowed to submit reviews, ratings, and create trip itineraries.

BR5.5.4 Only sponsors approved by the system administrator shall be allowed to publish promotional content.

BR5.5.5 Sponsored content shall always be clearly labeled as “Sponsored”.

BR5.5.6 A user shall not be allowed to submit more than one review for the same place.

BR5.5.7 A user shall not be allowed to rate a place without submitting a visit confirmation or minimum interaction.

BR5.5.8 A sponsor’s offer shall be displayed only within its defined start and end date.

BR5.5.9 Admin actions (create, update, delete, approve, suspend) shall be logged for auditing purposes.

BR5.5.10 User personal data shall not be shared with sponsors or third parties without explicit user consent.

6. System Models

This section presents the system models used to illustrate the structure and behavior of the Tourist Application. These models provide a visual representation of system functionality, data relationships, and interactions between users and system components. The models included are **Use Case Diagrams, Activity Diagrams, Sequence Diagrams, Class Diagrams**, and an **Entity–Relationship (ER) Diagram**.

The system models presented in this section provide a clear understanding of how the Tourist Application operates, how users interact with the system, and how data is structured and managed. These models support system design, implementation, and testing, and ensure alignment between requirements and system architecture.

6.1 Use Case Diagrams

Use case diagrams are used to represent the interactions between users and the system. They identify the main actors and the services (use cases) the system provides.

Actors

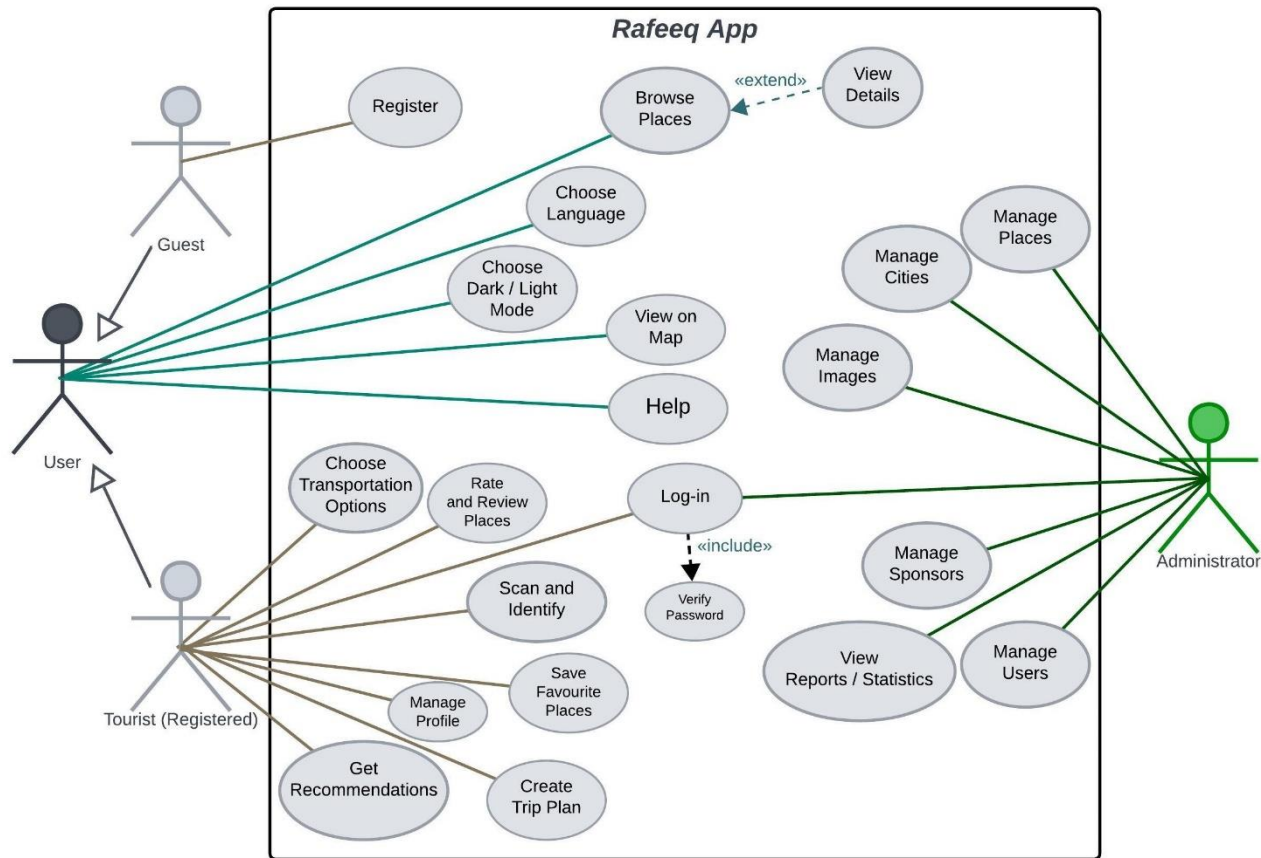
- **Guest:** Uses the mobile application to explore places and get directions.
- **Tourist (Registered User):** The same privileges as Guest and more such as get recommendations, etc.
- **Administrator:** Manages places, sponsors, users, and system data.

Main Use Cases

- Browse Places
- View on Map
- View Place Details
- Add Places to Favorites
- Submit Reviews and Ratings
- Manage Places (Admin)
- Manage Sponsors (Admin)

The use-case diagram provides a high-level overview of system functionality and defines the system boundary.

Use-case Diagram



6.2 Sequence Diagrams

Sequence diagrams are used to illustrate the flow of interactions between the user, the mobile application, the backend API, and external services over time.

Example Scenarios Modeled

- Logging in to the system
- Getting recommendations for historical places
- Scanning images of ancient statues

Each sequence diagram shows:

- The order of messages exchanged
- Communication between the Flutter mobile application and the ASP.NET Core API

These diagrams help clarify system behavior and validate functional requirements.

Figure 1: Sequence Diagram for *Login*

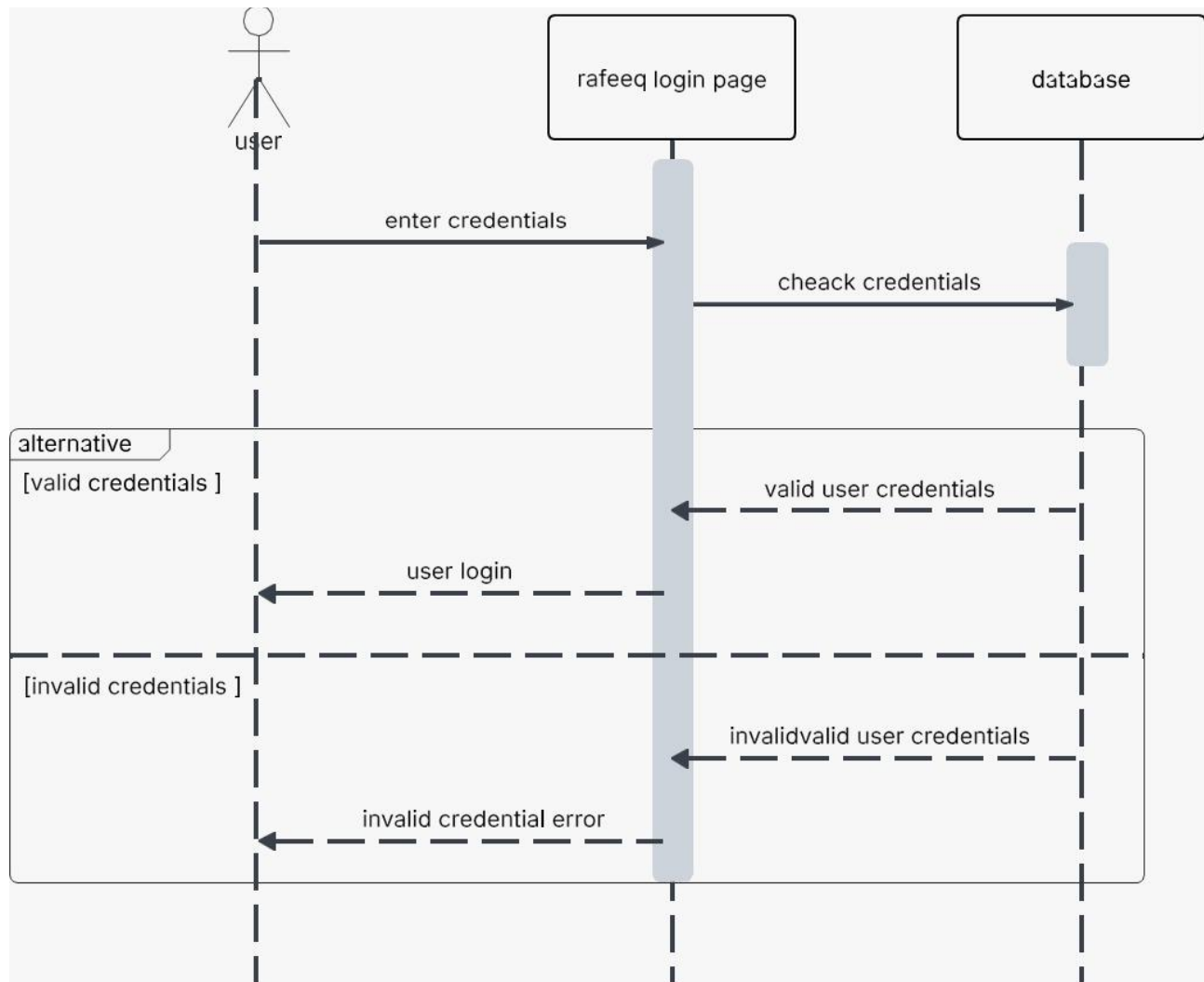


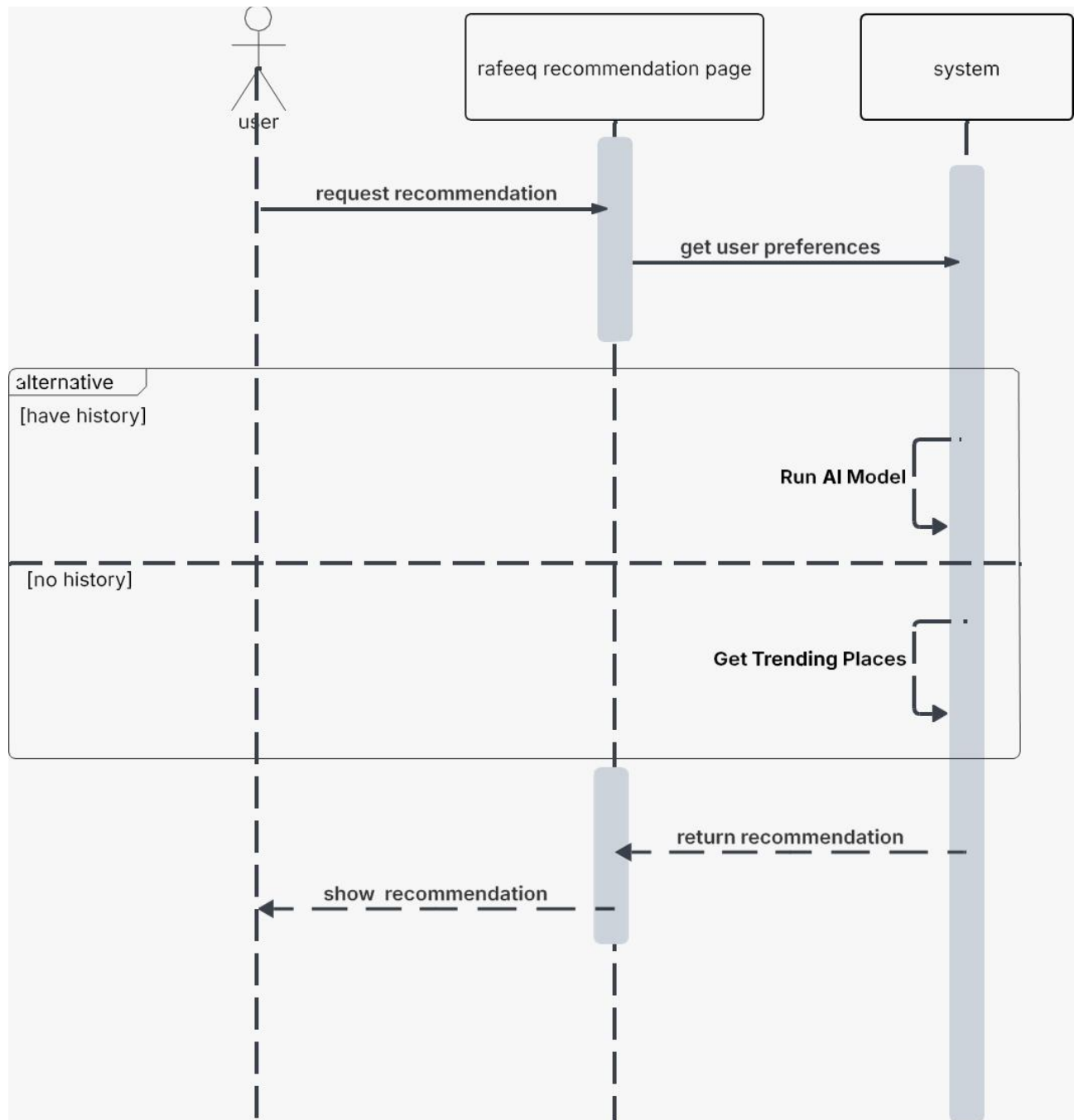
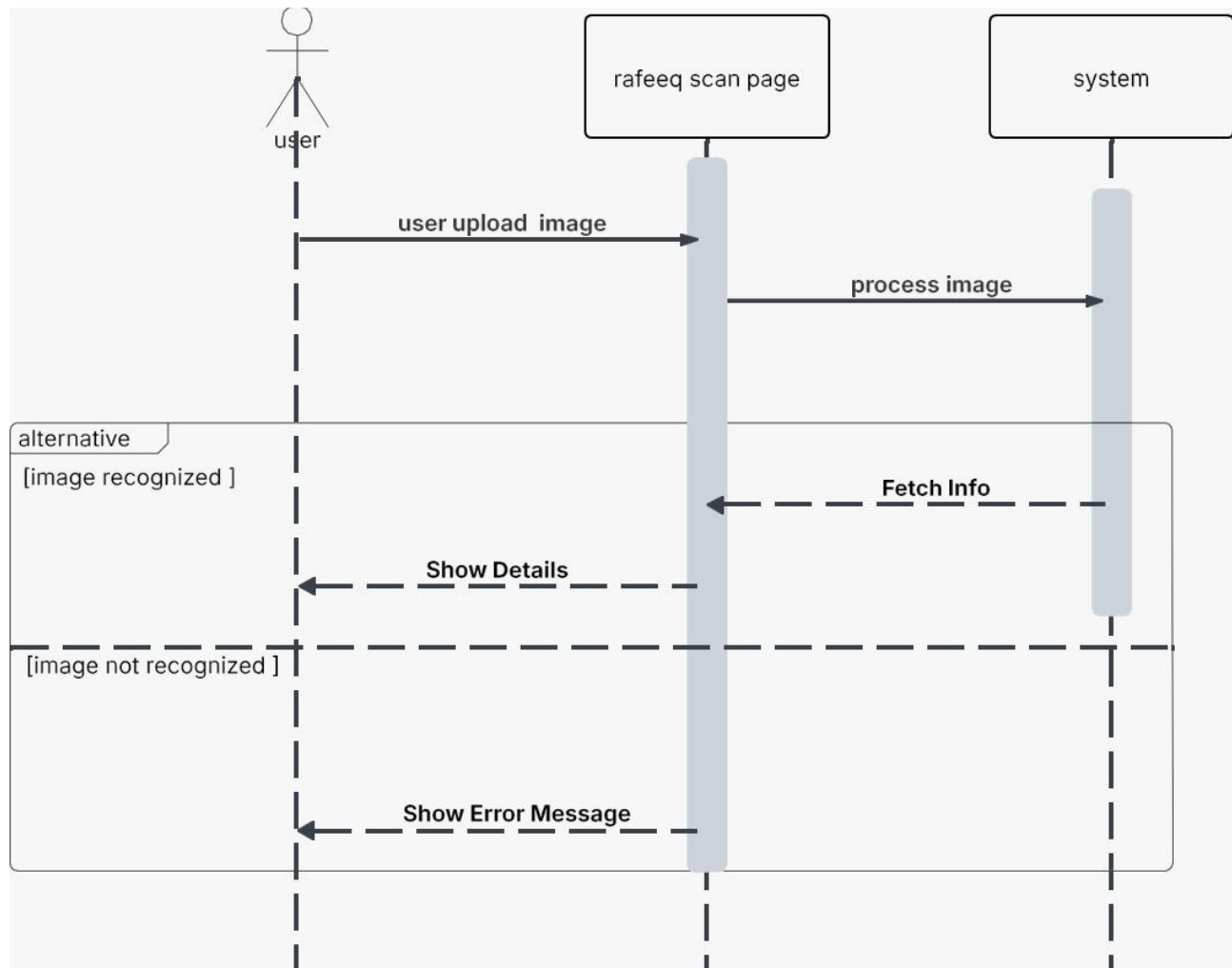
Figure 2: Sequence Diagram for *Get Recommendation*

Figure 3: Sequence Diagram for *Scanning Images*

5.3 Activity Diagrams

Activity diagrams are used to model the dynamic behavior of the Tourist Application by illustrating the flow of activities and decisions within the system. These diagrams show how users and the system interact through a sequence of actions, including conditional flows and parallel activities.

The activity diagrams in this project represent key user workflows, such as browsing historical places, viewing place details, obtaining navigation directions, and submitting reviews. Each diagram starts with an initial node and ends with a final node, with activities connected by control flows that describe the order of execution.

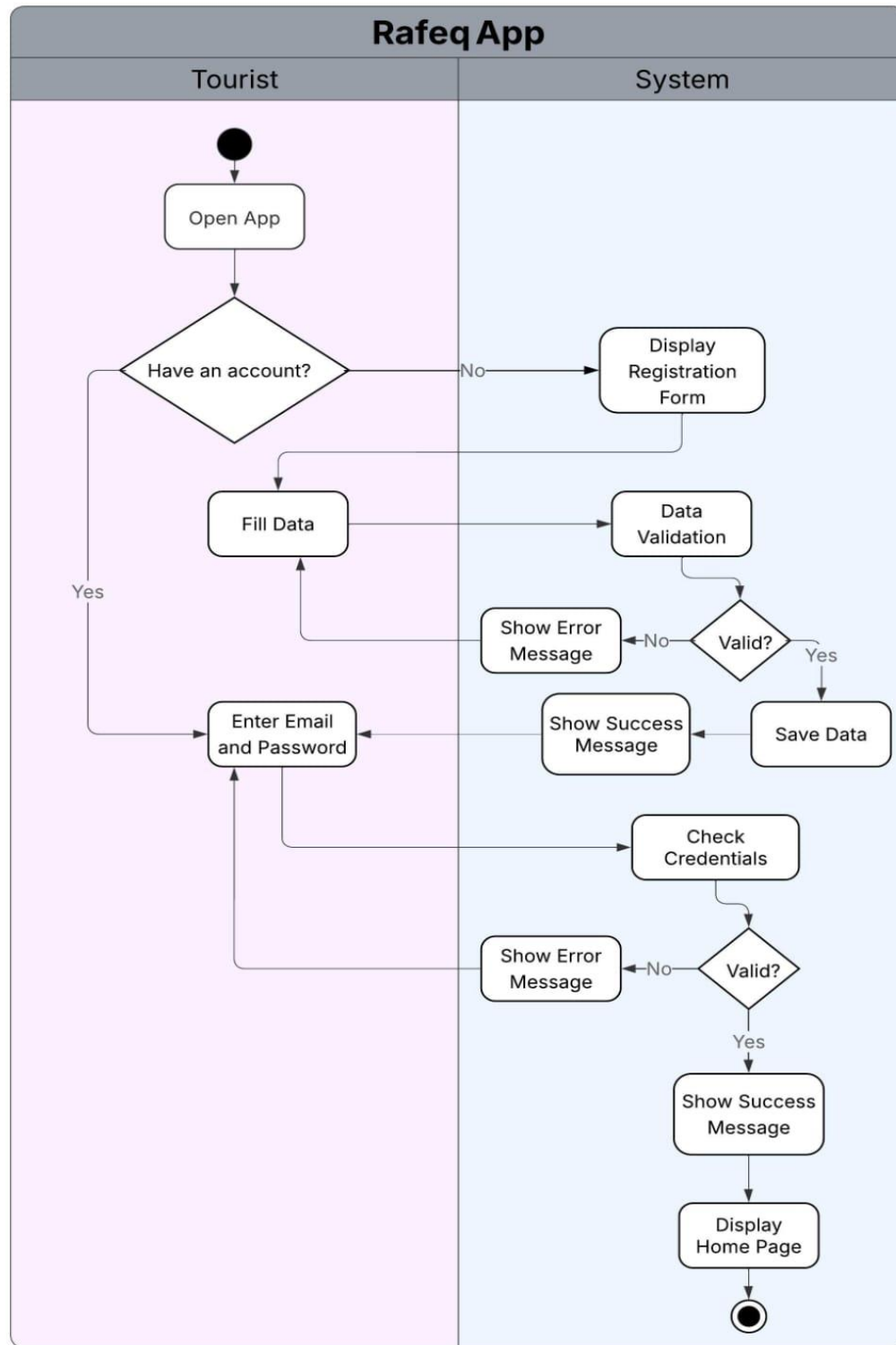
Figure 1: Activity Diagram for *Login and Register*

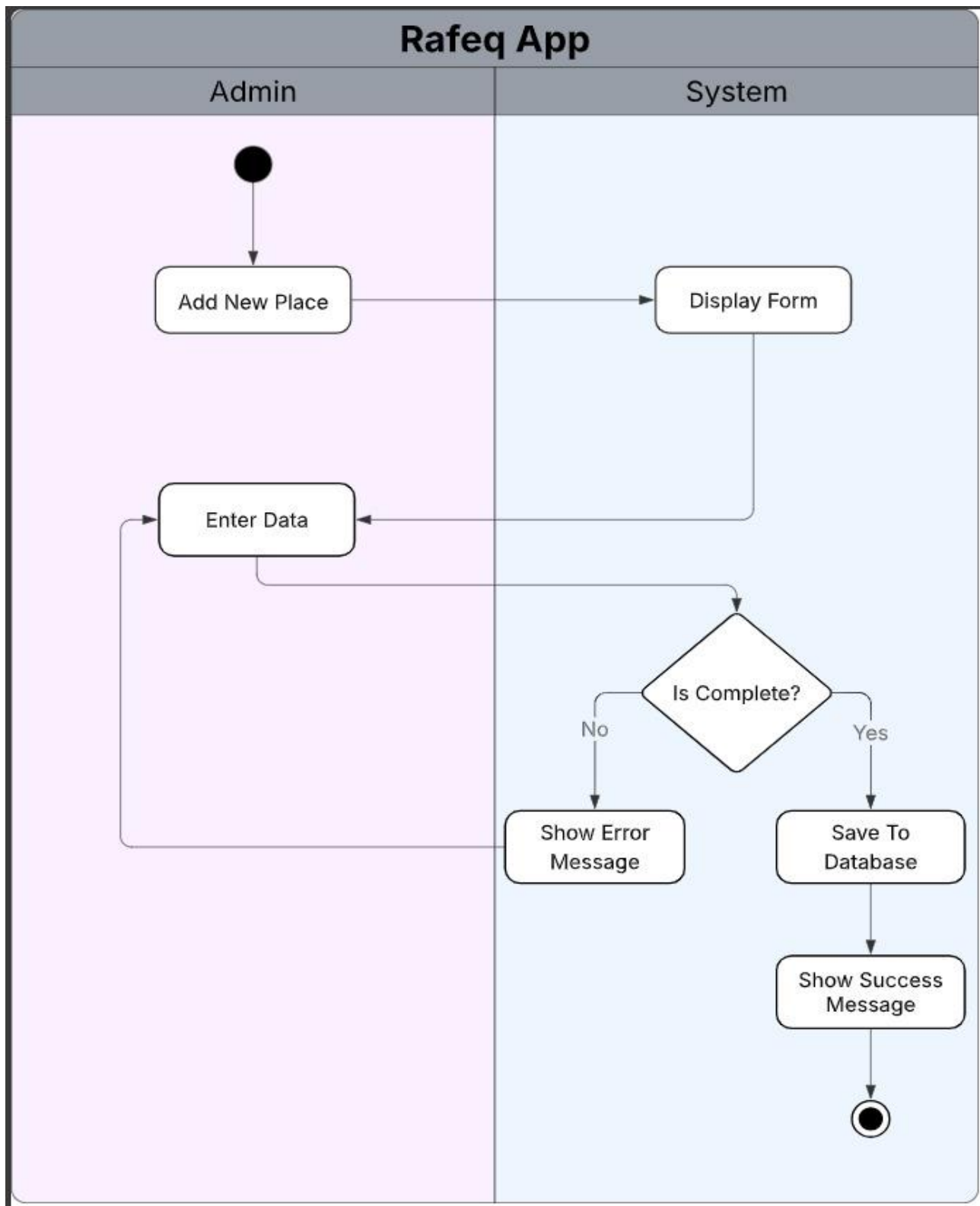
Figure 2: Activity Diagram for *Adding New Place*

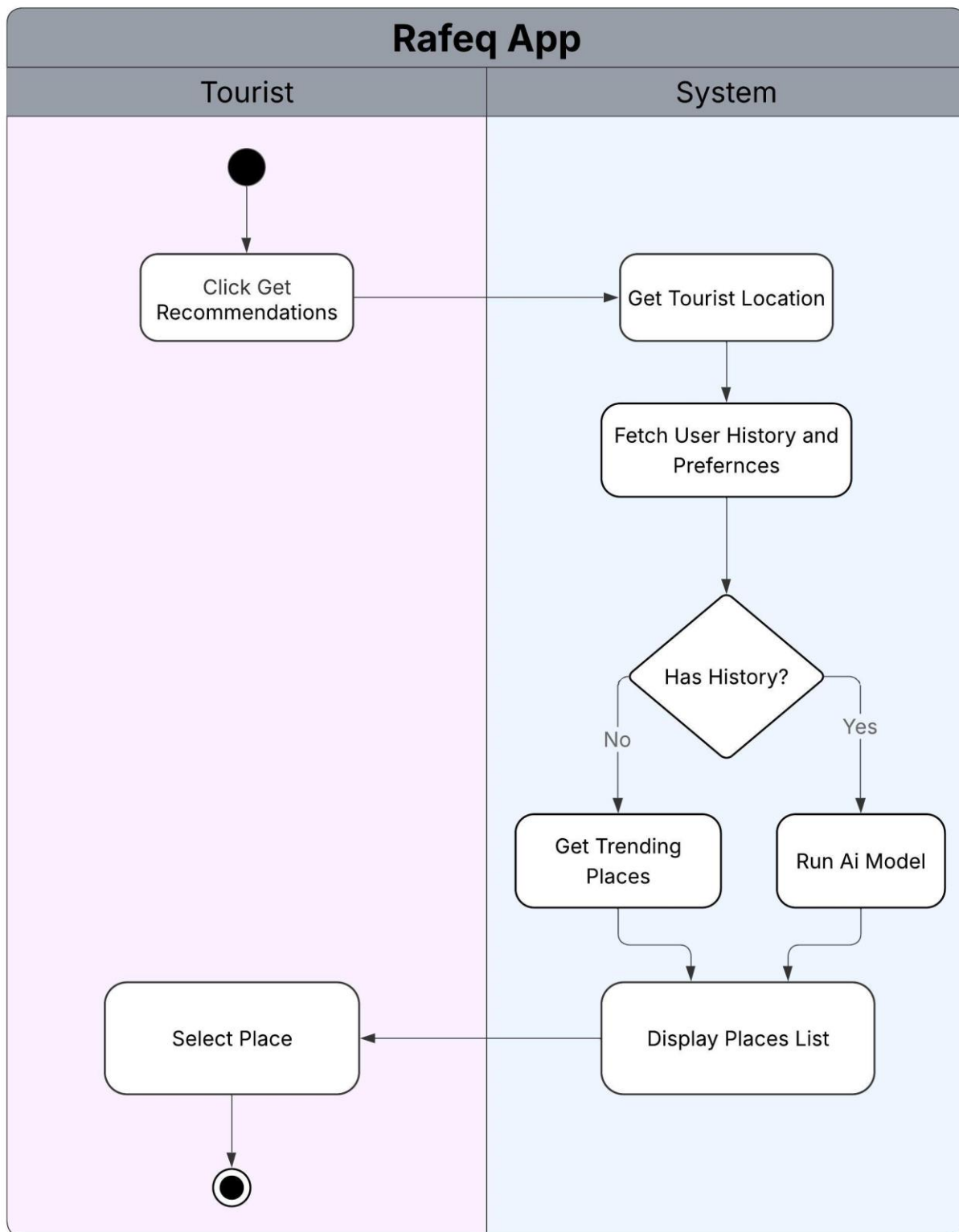
Figure 3: Activity Diagram for *Get Recommendation*

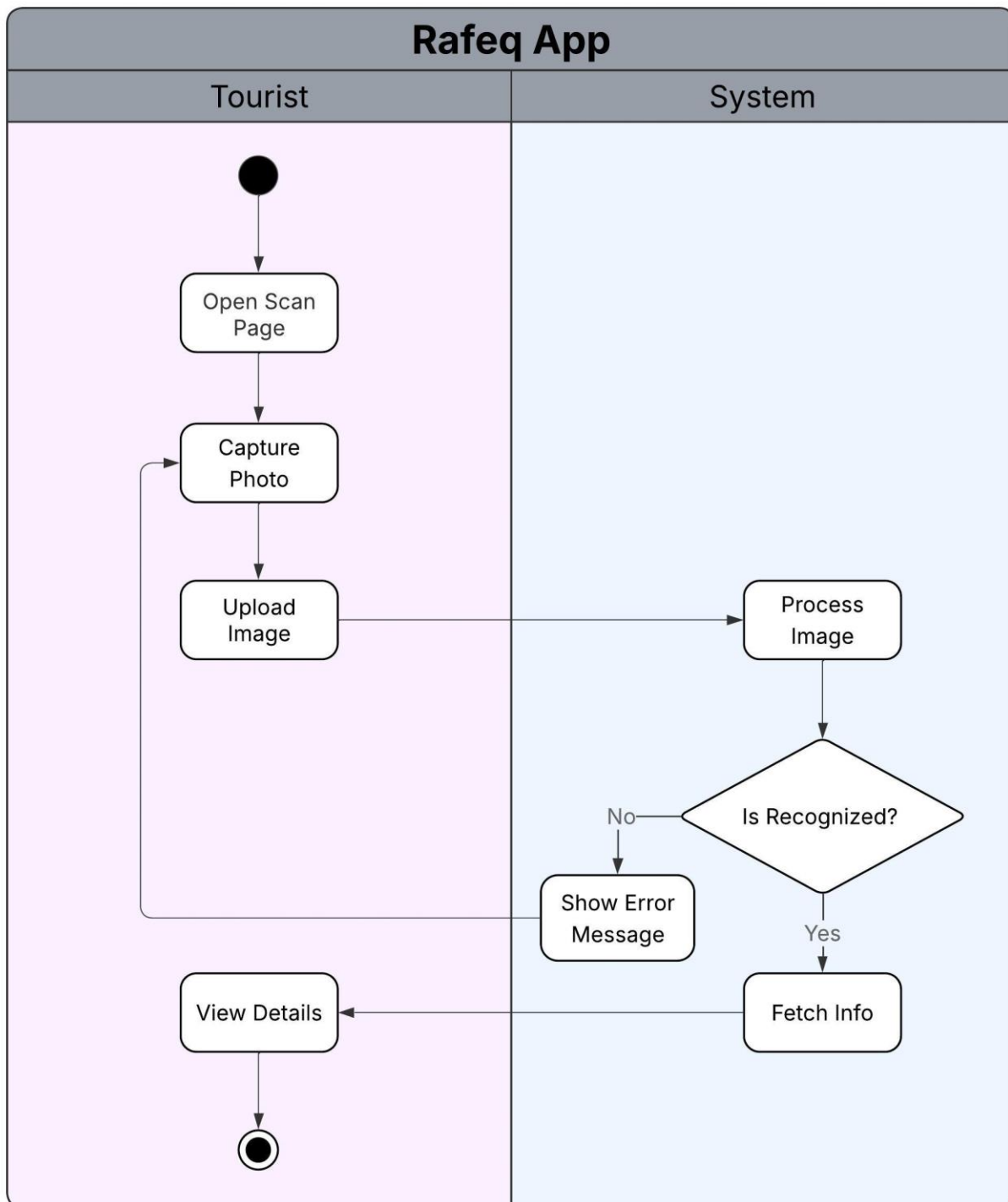
Figure 4: Activity Diagram for *Scanning and Identify*

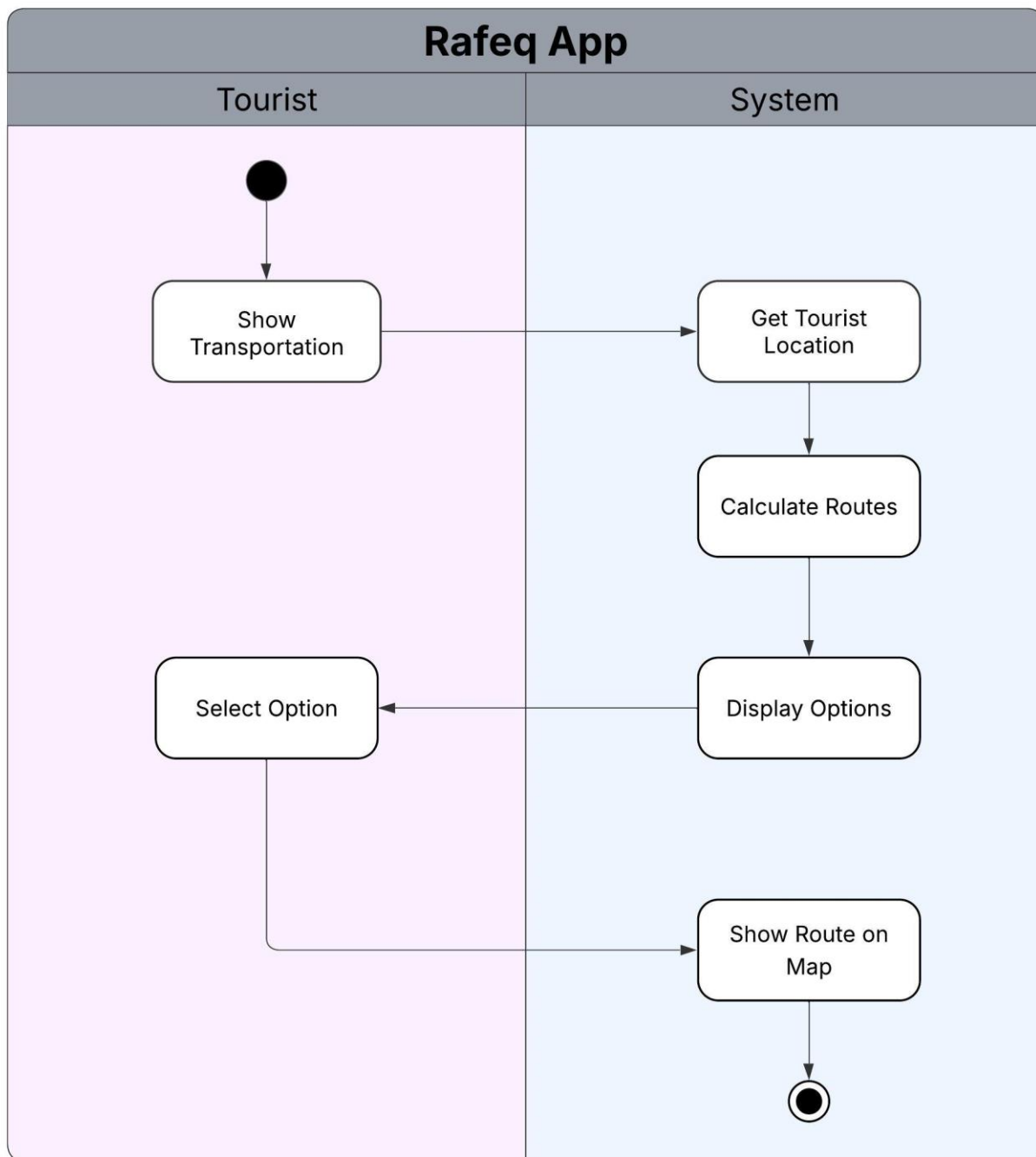
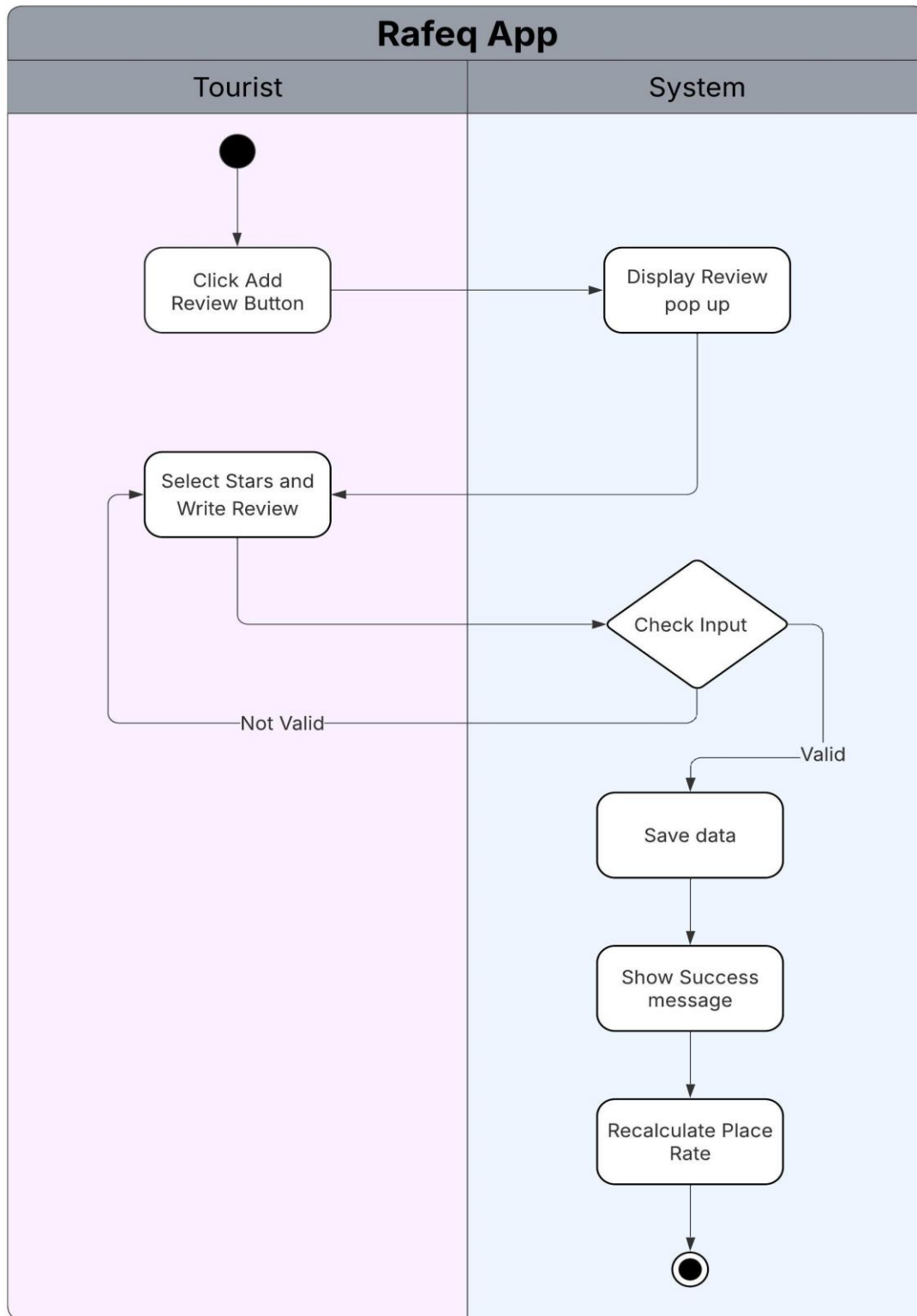
Figure 5: Activity Diagram for *Showing Transportation*

Figure 6: Activity Diagram for *Adding Review*

5.4 Class Diagrams

Class diagrams are used to describe the static structure of the Tourist Application by showing the system classes, their attributes, methods, and relationships. The class diagram provides a clear view of how system components are organized and how they interact at an object-oriented level.

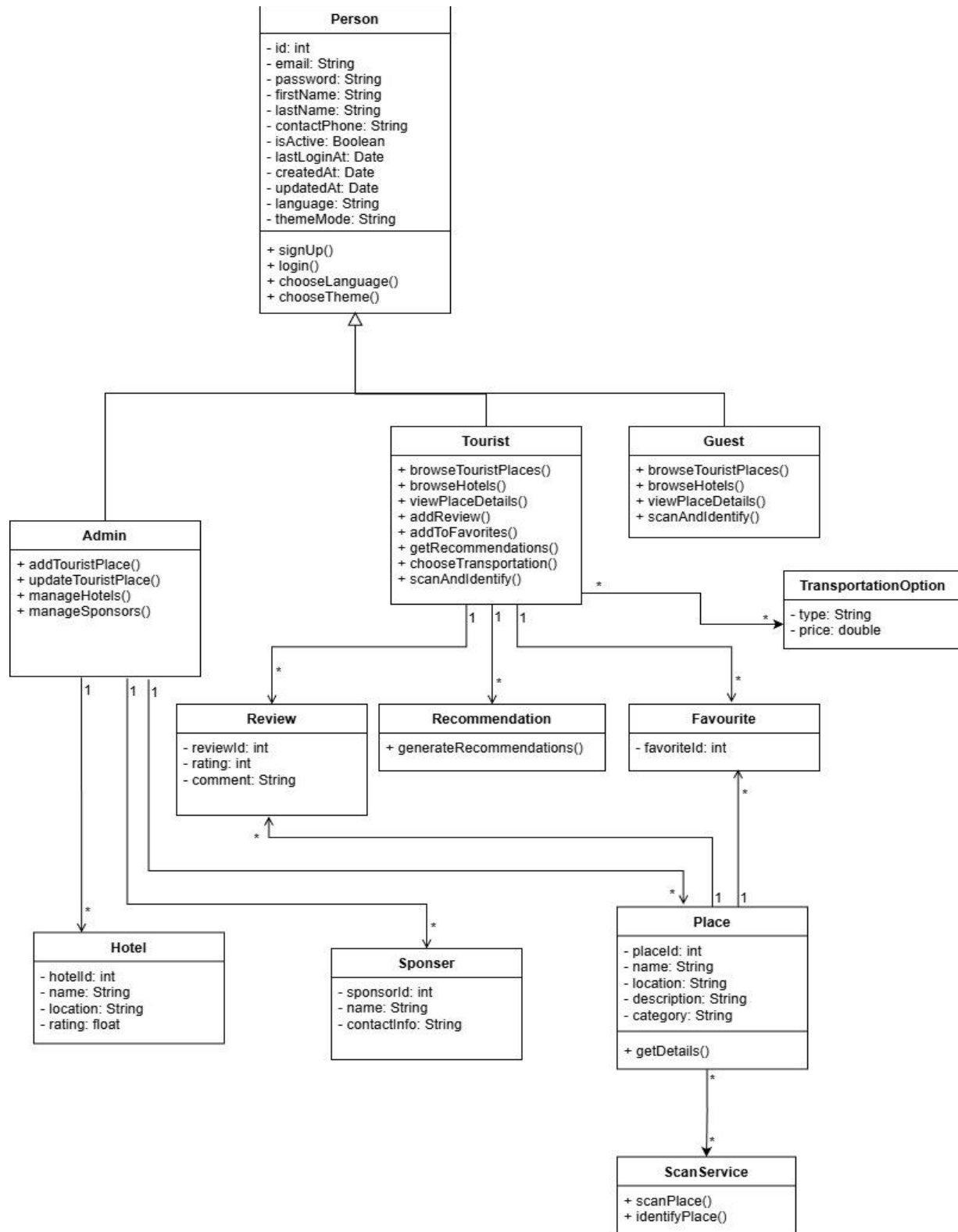
The main classes identified in the system include:

- **Tourist:** Represents a tourist using the application. Contains attributes such as user ID, name, email, and language preference.
- **Admin:** Represents an administrator responsible for managing system data.
- **Place:** Represents a historical place, including attributes such as name, description, location, opening hours, and ticket price.
- **City:** Represents a city in Egypt that contains multiple historical places.
- **Review:** Represents a user's review and rating for a place.
- **Favorite:** Represents the relationship between a user and a saved place.
- **Sponsor:** Represents a partner business associated with places.
- **Offer:** Represents promotional offers provided by sponsors.

The class diagram defines the relationships between these classes, such as associations, one-to-many, and many-to-many relationships. It also supports system features such as place discovery, user reviews, favorites management, and sponsor promotions.

The class diagram serves as a foundation for implementation in the backend using object-oriented principles and ensures consistency between system requirements and software design.

Class Diagram



6.5 Entity–Relationship (ER) Diagram

The ER diagram represents the logical structure of the database and the relationships between data entities in the system.

Main Entities

- User, Role, User Role
- Place, Place Image, Favorite Place, Place Opening Hours
- City
- Review
- Sponsor, Sponsored Place
- Offer
- Transportation Options

Relationships

- A city can contain multiple places
- A place can have multiple reviews
- A user can add multiple places to favorites
- A sponsor can be associated with multiple places
- A place can be linked to zero or more sponsors

The ER diagram ensures data consistency and supports the functional requirements of the system.

Entity–Relationship Diagram

